

Performance Testing

Date	13 July 2026
Team ID	N/A
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter (HTTP request sampler against the local FastAPI backend)
Type of Testing	Load Testing (steady concurrent users) and Spike Testing (sudden burst on the
Target Module / API	POST /api/financial/loans/{id}/analyze and POST /api/negotiation/generate - the
Test Environment	Local (uvicorn dev server on 127.0.0.1:8000, SQLite database)
Test Date	13 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load: normal usage	20	60	All requests succeed, avg response < 2s
2	Load: analyze endpoint under concurrent loan submissions	50	120	Financial engine returns correct EMI/DTI/settlement
3	Spike: sudden burst on negotiation-letter generation	100	60	Gemini calls queue gracefully or fall back to rule-based
4	Stress: sustained high concurrency to find breaking	200	180	System degrades gracefully (higher latency) rather than

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.6s	Pass	Non-AI endpoints (loan analysis)
2	Response Time (Max)	< 5 seconds	3.2s	Pass	Max observed under the 100-user spike
3	Throughput (Req/sec)		48 req/sec	-	Measured on the /api/financial/loans/{id} analyze
4	Error Rate	< 1%	0.4%	Pass	A small number of timeouts occurred
5	CPU Utilization	< 80%	62%	Pass	Measured on the local dev machine
6	Memory Utilization	< 80%	58%	Pass	SQLite + SQLAlchemy

Step 4: Observations & Analysis

Key Findings:

The rule-based financial-analysis endpoints (EMI/DTI/settlement scoring) are fast and lightweight, consistently responding well under the 2-second target even at 50 concurrent users, since they only perform in-process arithmetic over SQLite rows. The negotiation-letter endpoint is the main latency driver, since its response time is bounded by the external Gemini

Bottlenecks Identified:

Under the 200-user stress scenario, a small number of requests to the negotiation endpoint timed out while waiting on the Gemini API, and SQLite's single-writer behavior added minor queuing delay for simultaneous loan-update requests.

Optimization Steps Taken:

Confirmed the rule-based fallback letter generator kicks in immediately on a Gemini timeout so users still get a usable result instead of an error; verified database sessions are properly closed per-request (via FastAPI's dependency injection) to avoid connection buildup under load.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing output):

